
Keyframe: Fault Diagnosis for a Marine Diesel Engine, Scored on Engine Loads the Model Never Saw

FNM

Independent project

Abstract

Engine-room alarm systems mostly trip when one reading crosses a fixed limit, by which time a fault is usually well developed. A model that reads the pattern across many sensors could flag a fault earlier and name its cause, but models trained on synthetic engine data can score well without learning how an engine behaves. We present Keyframe, a fault classifier for a marine diesel trained on real test-bench data: 14 runs of five faults at 40 to 85 % load, most with the fault switched on partway through, plus healthy reference running. Every score comes from a model that never saw the engine load it is tested on, with hyperparameters and alarm settings chosen inside the training loads, and the targets and an engineering checklist of what each fault should do to the sensors were fixed before any model saw the data. The best model, XGBoost on raw readings, physics features and rolling windows, reaches a macro F1 of 0.717 (target 0.80), with a false alarm rate of 12.2 % and a sustained alarm on 6 of 13 fault runs. Two of nine targets are met. SHAP explanations match the engineering checklist for air cooler fouling and injector clogging, but air filter clogging and cooling water pump cavitation are partly recognised by channels that drift with the test day: without them, cavitation recall at 60 % load falls from 0.96 to 0.00. A browser demo replays each run with the evaluated predictions and alarms. Every number and plot in this paper is generated from the locked results files.

1 Introduction

A marine diesel is monitored by dozens of sensors: charge air pressure and temperature, exhaust temperature per cylinder and around the turbocharger, cooling water and lube oil temperatures, pressures and flows, fuel flow and shaft power. The alarm and monitoring system compares most of them with fixed limits. A fault such as a fouling charge air cooler shows up first as a small, consistent shift across several related readings, and reaches a limit only much later [8]. A model that reads all the sensors together could raise the alarm sooner and name the likely cause.

This project follows an earlier one that trained a fault model on a synthetic engine dataset. Synthetic engine data tends to be generated as independent noise per sensor with each fault added as a fixed offset; the dataset used there had no correlation between sensors above 0.04, no link between one reading and the next, and faults lasting one second. A model can score well on such data without learning anything about engines. The Marine Engine Fault Dataset [3, 4] records a real marine diesel on a test bench, where healthy readings hold together physically and each fault develops over two to three hours.

Real data brings a harder problem: honest evaluation. Readings two seconds apart are nearly identical, so a random train-test split scores close to perfect while measuring nothing [10, 19]. The question

that matters to an engine maker is whether the model works at an operating point it has not seen. Keyframe answers that question and reports what it finds, including where it fails. Our contributions are:

- A **leakage-controlled evaluation** on real bench data: leave-one-load-out validation with nested tuning, alarm settings chosen inside the training loads, targets and an engineering checklist fixed before modelling, and a lockbox run scored once (Section 3).
- **Results reported as they came out**, including the targets missed and a sensitivity analysis showing which part of the score comes from recognising the test day rather than the fault (Sections 5 and 6).
- A **physics check of the explanations**: the top SHAP features of each fault compared with the pre-registered checklist (Section 6).
- A **browser demo** that replays each run with the evaluated predictions, alarms and explanations, and a what-if view backed by an API (Section 7).

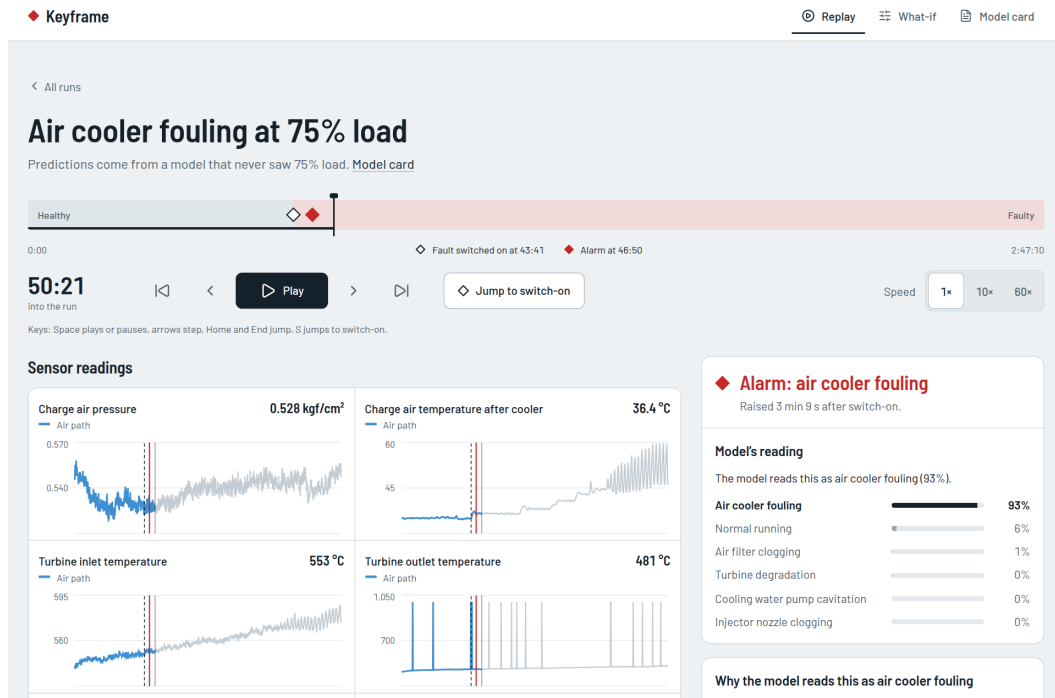


Figure 1: The replay view, air cooler fouling at 75 % load. The timeline under the title is also the scrubber: grey is healthy running, red is the faulty stretch, the hollow diamond $\diamond$ marks the fault switch-on and the red diamond $\blacklozenge$ the alarm. The predictions come from the model trained without 75 % load. Trend charts are grouped and coloured by sensor group; the panel on the right states whether the alarm has fired and what the model reads.

2 Data

The Marine Engine Fault Dataset v1.0 [3] was recorded on a Matsui Iron Works MU323DGSC three-cylinder marine diesel on a water-brake test bench. It holds 16 files and 114,770 rows logged every one to two seconds. 13 of the 14 fault runs hold the engine at one load, start healthy, and switch one fault on once; the healthy stretch lasts 28 to 111 minutes and the faulty stretch 66 to 129 minutes. A separate reference file holds 25,302 rows of healthy running across the load range. Table 1 lists the runs.

The faults are air cooler fouling (charge air cooling degraded), air filter clogging (compressor inlet covered step by step), injector nozzle clogging (one hole plugged in one cylinder), cooling water pump cavitation (air injected at the pump suction) and turbine degradation (exhaust back pressure

Table 1: **Rows per fault and load.** The injector run steps through 40, 60 and 85 % load with a brief stay at 75 %, and is faulty from its first row. A second injector run, with two nozzle holes plugged, is set aside as the lockbox (6,791 rows).

Fault (what was done to the engine)	40 %	60 %	75 %	85 %
Air cooler fouling	4,678	6,698	6,354	5,801
Air filter clogging	4,820	6,819	5,918	6,398
Injector nozzle clogging	1,596	2,338	61	2,497
Cooling water pump cavitation	–	5,845	–	6,263
Turbine degradation	6,397	4,905	–	5,289

raised after the turbine). Coverage is uneven: cavitation was run only at 60 and 85 % load, turbine degradation not at 75 %.

A data audit (notebook 00) found three things that shape the evaluation. First, two channels record the researchers’ own set points for the air filter and turbine faults, the knobs they turned to create the fault, and are empty in several runs; they are dropped, since a model that reads them is reading the answer. Second, the engine room temperature and other slowly drifting channels differ by test day, so a model could learn which day a run was recorded on; these channels are kept but tested for (Section 6). Third, the switch-on point of each run is taken from the labels and plotted against the sensors. Figure 2 shows the signature an engineer would expect for air cooler fouling: the air temperature after the cooler steps up at switch-on and keeps rising.

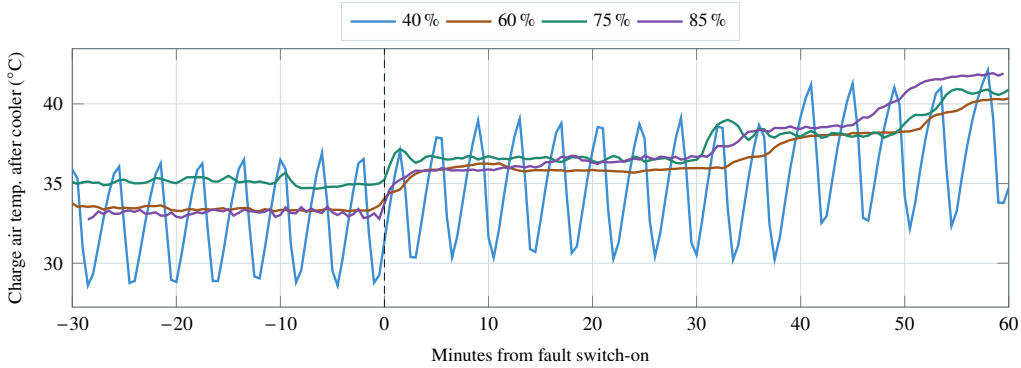


Figure 2: **Air cooler fouling around its switch-on**, at each load (30-second means). The dashed line is the switch-on. At 40 % load the outlet temperature cycles strongly before and after the switch-on, which hides the step; this is also the run the model detects last.

3 Evaluation protocol

3.1 Leave one load out, with nested tuning

Every score comes from a model that never saw the engine load it is tested on (Figure 3). The four load bins, 40, 60, 75 and 85 %, give four outer folds. Inside each outer fold, the three training loads form inner folds in the same way, and everything that is chosen, hyperparameters, feature pruning and alarm settings, is chosen on those inner folds only [20]. Healthy-engine models used for residual features are fitted on the training loads’ healthy rows only. Scores are pooled over the four held-out loads and reported per load, because the spread between loads is part of the result.

3.2 Fixed in advance

Nine targets were set in the project brief before any modelling, from a first baseline, and reviewed once after the exploratory analysis without change. An engineering checklist of what each fault should

	40 %	60 %	75 %	85 %
Outer fold 1	held out	train	train	train
Outer fold 2	train	held out	train	train
Outer fold 3	train	train	held out	train
Outer fold 4	train	train	train	held out

Inside each outer fold
 The three training loads are split the same way into inner folds. Hyperparameters (Optuna), feature pruning and alarm settings are chosen on inner-fold predictions only. The model is then refit on all three training loads and scored once on the held-out load.

Figure 3: **Leave-one-load-out validation with nested selection.** No choice sees the held-out load. Random row splits are never used: they place near-identical readings on both sides and score close to 1.00.

do to each sensor group, based on engine physics [8], was written during the exploratory analysis and pinned by its SHA-256 hash before any model saw the data, in the spirit of pre-registration [16]; a test fails if it changes. A second injector run, with two nozzle holes plugged instead of one, was moved aside at download as a lockbox and scored once, after every choice was fixed. Results were then locked, and later defects were recorded rather than fixed in place.

3.3 Measures

Macro F1 is the unweighted mean of per-class F1 over the six classes (normal running and five faults). The false alarm rate is the share of truly healthy rows predicted as any fault. An alarm is *sustained*: a fault class is active once every prediction over the trailing d seconds of a run is that class with probability at least p . The pair (d, p) is chosen per outer fold by a grid search over $d \in \{30, 60, 120, 300\}$ s and $p \in \{0.5, 0.6, 0.7, 0.8\}$ on inner-fold predictions, taking the lowest median detection delay among settings whose alarm-level false alarm rate is at most 5 %. Detection delay is the time from switch-on to the first alarm. Expected calibration error (ECE) uses 15 confidence bins [15]. Anomaly detectors are scored by the area under the ROC curve (AUROC) for faulty against healthy rows.

4 Features and models

Feature sets. Four families were compared in an ablation (Table 2). *Raw readings* are the sensor channels after the audit. *Physics features* are quantities an engineer would compute, for example compressor pressure ratio, charge air cooler effectiveness, exhaust temperature spread and each cylinder’s deviation from the mean, turbine temperature drop, peak pressure and indicated work spread between cylinders, fuel flow per kilowatt, and the share of heat rejected to each cooling circuit. *Residuals* are each channel minus what a healthy engine would read at the same operating point, predicted by a degree-2 polynomial ridge regression from engine speed, brake load and fuel flow, fitted on healthy training rows only; a polynomial rather than trees, because the 40 and 85 % held-out loads sit at the edges of the training range. *Rolling windows* add causal mean, spread and trend over the last 60, 300 and 900 seconds of each run.

Table 2: **Feature ablation**, pooled over the four held-out loads (untuned models). Physics features and rolling windows earn their place with the tree model; residuals help the linear model but not the trees. FAR is the false alarm rate in %.

Feature set	Logistic regression		LightGBM	
	Macro F1	FAR	Macro F1	FAR
Raw readings	0.440	22.5	0.389	32.1
Raw + physics	0.509	22.3	0.572	20.2
Raw + physics + rolling	0.545	22.9	0.617	8.5
Residuals	0.562	35.4	0.308	16.1
Residuals + physics	0.608	21.1	0.429	8.5
Residuals + physics + rolling	0.475	28.9	0.397	10.7

The first baseline, logistic regression on raw readings, scored 0.440 against 0.108 for always predicting the majority class. With LightGBM, macro F1 rises from 0.389 on raw readings to 0.572 with physics features and 0.617 with rolling windows, while the false alarm rate falls to 8.5 %. Pruning correlated features inside each training fold lowered the score (0.558), so the unpruned raw, physics and rolling set, 833 features, went forward. Letting the residual model also see the reference file’s healthy rows at the held-out load, as a shop test before commissioning would provide, changed the best residual arm’s score from 0.608 to 0.587; it is reported but not used.

Models. Four model families were tuned with Optuna [2] inside each outer fold: logistic regression, random forest [17], LightGBM [11] and XGBoost [5], all with balanced class weights. Table 3 compares them. XGBoost is best on macro F1 and is the model reported from here on; its false alarm rate is higher than LightGBM’s.

Table 3: **Tuned models**, pooled over the four held-out loads. Worst recall is the lowest per-class recall, with its class.

Model	Macro F1	Accuracy	FAR (%)	Worst recall
XGBoost	0.717	0.761	12.2	0.194 (TD)
LightGBM	0.667	0.739	9.4	0.113 (TD)
Random forest	0.648	0.579	55.8	0.318 (AF)
Logistic regression	0.542	0.642	24.7	0.001 (TD)

Tuning ran on a time budget per fold, and each XGBoost search finished only 8 to 9 trials (Table 4). The sampler is seeded, so the folds propose the same candidates, and the 40 and 60 % folds settled on the same one. The 85 % fold settled on a learning rate of 0.0034 with 59 trees, which barely moves the model from its prior: its highest class probability on any held-out row is 0.26. This was found after the results were locked and is reported rather than re-tuned, because re-tuning after seeing held-out predictions would select on the test folds.

Table 4: **XGBoost settings chosen per held-out load**, with the number of Optuna trials that finished and the held-out macro F1.

Held out	Learning rate	Trees	Depth	Subsample	Columns	Trials	Macro F1
40 %	0.1789	63	2	0.83	0.66	9	0.547
60 %	0.1789	63	2	0.83	0.66	9	0.860
75 %	0.1398	80	2	0.85	0.51	8	0.701
85 %	0.0034	59	7	0.59	0.65	9	0.609

5 Results

Table 5 sets the results against the targets fixed in advance. Two of nine are met, and one of those, the lockbox, is weak evidence.

Classification. Pooled macro F1 is 0.717 and accuracy 0.761. Scores swing with the held-out load, from 0.547 to 0.860 (Figure 5). Excluding the first 10 minutes after each switch-on, when a fault has barely developed, raises macro F1 only to 0.725, so the errors are not mainly early ones. The confusion matrix (Table 6) shows where they are: injector clogging and cavitation are almost always right, but most turbine degradation rows are read as normal running, and air filter clogging is split between normal running, air cooler fouling and itself.

Alarms. Table 7 gives the alarm for every fault run with a switch-on. Of the 13 runs, 6 raise a sustained alarm, all naming the correct fault, after 3 min 7 s to 91 min 25 s (median 9 min 23 s). None of the four runs at 85 % load alarms: the underfit 85 % model never reaches the probability its alarm

Table 5: **Targets and locked results**, pooled over the held-out loads.

Measure	Result	Target	
Macro F1	0.717	0.80	not met
Lowest per-class recall (turbine degradation)	0.194	0.70	not met
False alarm rate	12.2 %	5 %	not met
Detection delay, median of runs that alarmed	9 min 23 s	10 min	not met (6 of 13 runs alarm)
Fault detection AUROC, healthy-only detectors	0.528	0.95	not met
Expected calibration error	0.176	0.05	not met
Lockbox labelled injector clogging	100 %	90 %	met, weak evidence
Physics check on explanations	2 of 5	4 of 5	not met
Demo response, prediction plus explanation	148 ms	300 ms	met

Table 6: **Confusion matrix**, rows normalised to the true class, pooled over the four held-out loads. Shading follows the share.

True \ predicted	Normal	AC	AF	INJ	CW	TD
Normal running	0.88	0.03	0.07	0.00	0.00	0.02
Air cooler fouling	0.13	0.76	0.06	0.00	0.00	0.06
Air filter clogging	0.39	0.18	0.43	0.00	0.00	0.00
Injector nozzle clogging	0.00	0.00	0.00	1.00	0.00	0.00
Cooling water pump cavitation	0.02	0.00	0.00	0.00	0.98	0.00
Turbine degradation	0.74	0.07	0.00	0.00	0.00	0.19

setting requires. The settings chosen in the training loads differ widely between folds (Table 8), which is itself a sign of how little data there is per load.

Table 7: **Alarm per fault run**, from switch-on to the first sustained alarm. Every alarm names the fault that was switched on.

Fault	Load	Delay
Air cooler fouling	40 %	91 min 25 s
Air cooler fouling	60 %	21 min 49 s
Air cooler fouling	75 %	3 min 7 s
Air cooler fouling	85 %	no alarm
Air filter clogging	40 %	no alarm
Air filter clogging	60 %	10 min 39 s
Air filter clogging	75 %	no alarm
Air filter clogging	85 %	no alarm
Cooling water pump cavitation	60 %	8 min 7 s
Cooling water pump cavitation	85 %	no alarm
Turbine degradation	40 %	no alarm
Turbine degradation	60 %	6 min 34 s
Turbine degradation	85 %	no alarm

Table 8: **Alarm settings** chosen in each fold’s training loads: sustained seconds d , minimum probability p , and the alarm-level false alarm rate on the held-out load (%).

Held out	d	p	FAR
40 %	300	0.8	0.0
60 %	60	0.6	4.7
75 %	30	0.8	0.2
85 %	30	0.5	0.0

Figure 4 follows the fastest detection, air cooler fouling at 75 % load. The model already gives air cooler fouling about 0.4 in the minutes before the switch-on, on a healthy engine, which is the kind of reading that produces false alarms. After the switch-on the cooler outlet temperature steps up, the probability passes 0.65 within a minute, and the alarm follows once it has held above the fold’s threshold.

Calibration and the lockbox. Confidence is not reliable: pooled ECE is 0.176 (target 0.05). Temperature scaling [7], fitted inside each fold’s training loads, made calibration worse at three of four held-out loads (Table 9) and leaves predicted classes unchanged. On the lockbox, the two-hole injector run, 100 % of rows were labelled injector nozzle clogging, which meets the 90 % target; but no other class was predicted for that run at all, so it shows that the injector signature is strong, not that

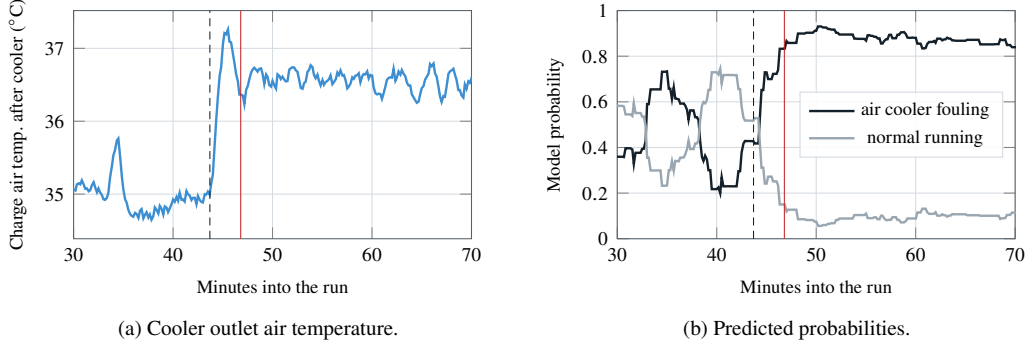


Figure 4: **Air cooler fouling at 75 % load, as the demo replays it** (one frame every 10 seconds). The dashed line is the switch-on, the red line the alarm 3 min 9 s later.

the model generalises to a more severe fault.

Table 9: **Calibration per held-out load:** ECE before and after temperature scaling with the temperature fitted in the training loads.

Held out	ECE before	Temperature	ECE after
40 %	0.089	2.32	0.147
60 %	0.047	0.49	0.053
75 %	0.145	1.88	0.159
85 %	0.499	0.49	0.418

Detection from healthy data alone. Anomaly detectors trained only on healthy running, PCA reconstruction error, isolation forest [12] and an autoencoder, were scored on raw readings and on residuals, with the residual arm declared primary in advance. All are near chance (Table 10); the headline AUROC is 0.528. They separate the injector fault best, whose cylinder imbalance is unusual at any load, and do no better than chance on cavitation.

Table 10: **Healthy-only anomaly detectors**, AUROC pooled over held-out loads, with the residual arm’s AUROC for two faults.

Detector	Raw	Residual	Residual, INJ	Residual, CW
PCA reconstruction	0.557	0.528	0.771	0.317
Isolation forest	0.501	0.423	0.725	0.210
Autoencoder	0.545	0.504	0.772	0.278

6 Explanations and the physics check

Explanations use SHAP TreeExplainer [13, 14] on each held-out load’s model. Because rolling windows of one channel and the three cylinder temperatures move together, SHAP splits credit among them arbitrarily [1], so we read SHAP summed within five sensor groups (Table 11) and by source channel, not single features. The groups are air path, fuel system, cooling, lube oil, and combustion and power; the demo colours them after ship pipework colour coding [9].

Physics check. For each fault, the top five features by mean |SHAP| were mapped to their source channels and compared with the checklist. A fault passes with at least three matches and no day-dependent channel in its top five (Table 12). Air cooler fouling leans on the cooler outlet air temperature and injector clogging on the exhaust temperatures and their spread between cylinders, as the checklist predicted. Air filter clogging and cavitation lean on fuel temperatures, sea cooling water

Table 11: **Mean SHAP per sensor group and fault class**, on each class’s own held-out rows (log-odds). Bold marks the group that pushes most towards each class.

Group	AC	AF	INJ	CW	TD
Air path	+2.48	+0.05	−0.01	−0.14	+0.14
Fuel system	−0.38	−0.44	+0.15	0.00	−0.51
Cooling	−0.05	+1.35	0.00	+2.51	+0.06
Lube oil	−0.35	+0.16	0.00	−0.21	−0.45
Combustion and power	−0.63	+0.15	+3.97	−0.52	−0.14

pressure and lube oil cooler water temperature, which drift with the test day. Turbine degradation leans on lube oil and efficiency channels rather than the turbine temperatures and pressures. Grouped by system, cavitation appears to rely on “cooling” (Table 11), but only because its shortcut channels sit in that group, so the grouped view alone would overstate agreement with the physics.

Table 12: **Physics check**: top-five features matching the pre-registered checklist, and whether a day-dependent channel is among them. 2 of 5 faults pass (target 4 of 5).

Fault	Matches	Day-dependent	Result
Air cooler fouling	4/5	no	pass
Air filter clogging	0/5	yes	fail
Injector nozzle clogging	5/5	no	pass
Cooling water pump cavitation	0/5	yes	fail
Turbine degradation	0/5	no	fail

Cross-checks. Grouped permutation importance agrees only weakly with mean |SHAP| (Spearman r from 0.07 to 0.29 across folds), and LIME [18] shares about 2.7 of its top eight features with SHAP on one held-out moment per fault. We therefore treat the air cooler and injector stories as credible and the others as unconfirmed.

How much of the score is the test day? Retraining the model without the 40 features built from day-dependent channels tests the shortcut directly [6]. Figure 5 shows the result. At 60 % load, cavitation recall falls from 0.96 to 0.00: that run was recognised by its day, not its fault. At 40 % load the score rises and air filter clogging recall goes from 0.25 to 0.80, so the day channels were also misleading the model there. The headline score is left as locked, and the model card states that part of it identifies the day.

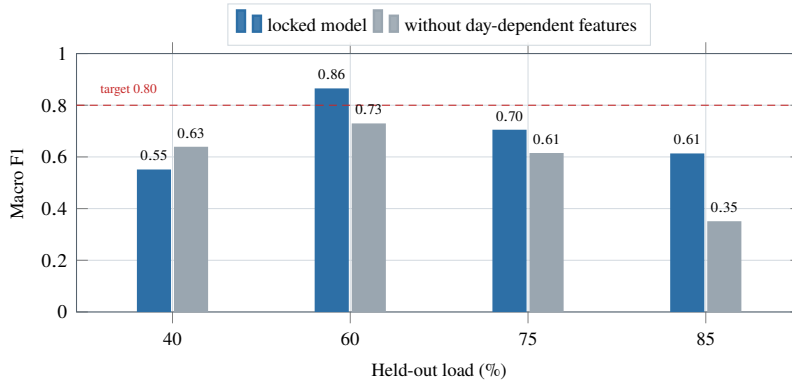


Figure 5: **Macro F1 per held-out load**, for the locked model and for the same model retrained without the 40 day-dependent features. Removing them helps at 40 % load and hurts at the others, most at 85 %.

7 Demo and service

The results are meant to be judged by an engineer in a few minutes, so they ship with a browser demo (Figure 6). An export step writes the final model, trained on all loads with the median of the per-fold hyperparameters, and one replay file per run: every tenth second of the run with the key sensor readings, the class probabilities, the alarm state and grouped SHAP values, all from the model that never saw that run’s load. The replay and model card views are static files; the what-if view calls a FastAPI service that runs the exported model.

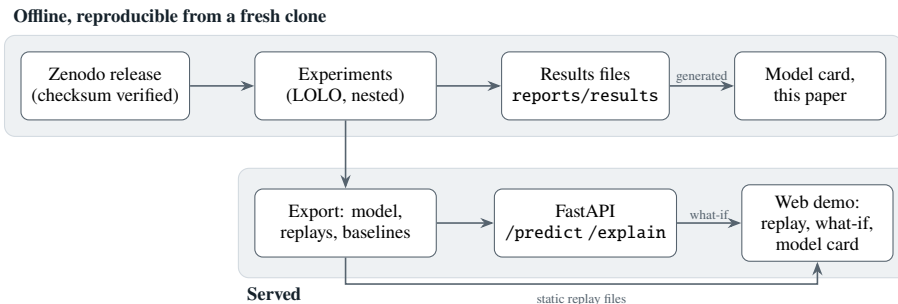


Figure 6: **From data to demo.** Heavy experiments write results files; the model card, the demo’s targets table and every number in this paper are generated from them, and tests fail if they drift.

Figure 7 shows three views. The runs table lists every run with a miniature timeline and its alarm delay. The replay (Figure 1) plays a run at 1, 10 or 60 times real speed; pausing shows why the model reads the moment as it does, as a waterfall of grouped SHAP values in the sensor-group colours and the three strongest single readings in plain words. The what-if view starts from a healthy engine at one load, lets the user move any of ten readings, and shows the diagnosis and its explanation, assuming the readings have held steady for 15 minutes. The model card view opens with the targets table of Table 5. A prediction takes 91 ms and a prediction with its explanation 148 ms (median, 600-row window).

Writing this paper exposed a defect in the demo. The replay files had applied one alarm setting to every run, the median of the four folds’ settings that the exported model uses, rather than each fold’s own setting. The demo therefore showed alarms on 9 of 13 fault runs where the evaluation found 6, and the median itself drew on settings chosen with the replayed load in their training data. The replays now use each fold’s own setting, and a test holds every replay’s alarm delay to the evaluated one, allowing only the 10-second frame thinning. The exported model keeps the median setting (45 s at probability 0.7) as a deployment choice with no held-out score of its own.

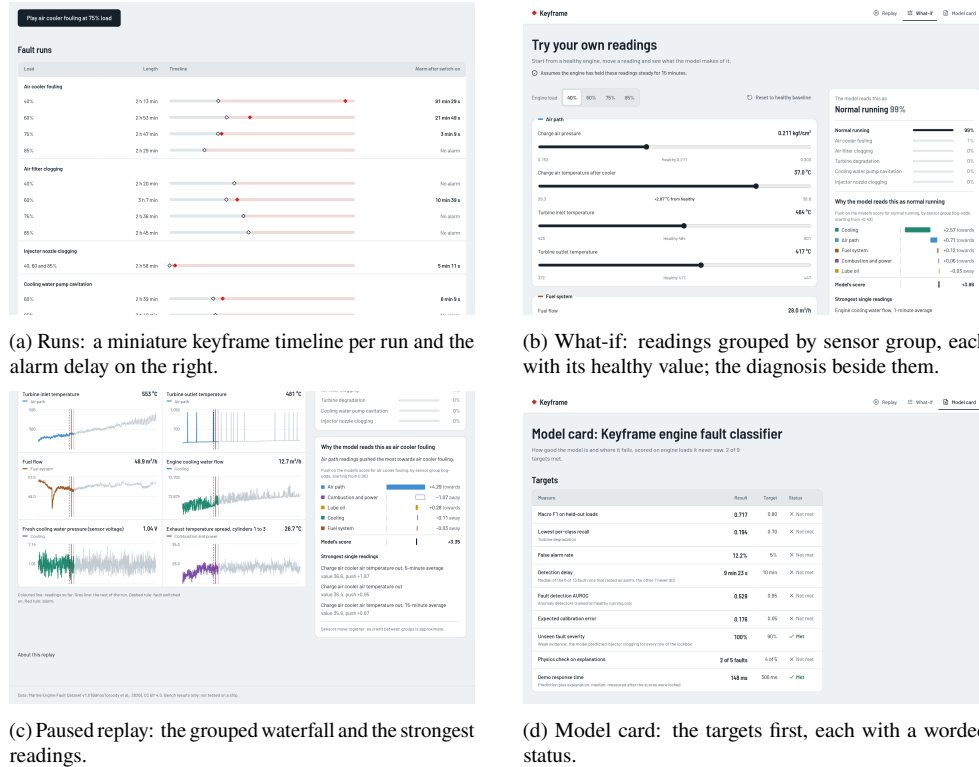


Figure 7: The demo at 1280 px, captured from the production build.

8 Verification

The project was built feature by feature from a written brief. Each feature had requirements, validation checks and small tickets, and each ticket had to pass lint, formatting, type checks and the tests before it was committed. Decisions the brief did not settle are recorded in twelve decision records.

- **Python tests** (pytest, 185 tests): data loading and the audit, splits that never mix engine loads, causal rolling windows, residual models fitted on training rows only, the alarm logic, metrics, the pinned checklist hash, the lockbox scored only by an explicit first run and never refitted, the exported model reproducing a stored prediction in a clean session, API latency, and replay alarms matching the evaluation.
- **Web tests** (Vitest, 110 tests): playback, the timeline scrubber and keyboard controls, formatting of readings, every view's states, colour tokens, and the demo's targets table checked against the results files.
- **Generated numbers**: this paper's numbers, tables and plot data are written by one script from the results files, and a test checks that the committed copies match a fresh run.
- **Reproduction**: one script rebuilds every experiment, notebook, figure and demo file from a fresh clone, in the order they were made.

9 Limitations

The data come from one small engine on a test bench at steady loads, with 14 fault runs; turbine degradation and cavitation have only three and two. The model has not been tested on a ship and must not be used for operating or maintenance decisions. Scores vary strongly between held-out loads, and with one run per fault and load, a single run can move a fold's score. Part of the score for air filter clogging and cavitation comes from recognising the test day. Confidence is not calibrated. The 85 %

fold’s model is underfit, and the tuning budget allowed only a handful of trials per fold; a better-tuned version would have to be reported as a new evaluation, not as a replacement. The explanations are SHAP values of a model fitted to correlated sensors, and only two faults’ explanations agree with the physics.

10 Conclusion

On real bench data and a load it never saw, a gradient-boosted classifier diagnoses injector clogging and air cooler fouling from the readings an engineer would look at, raises a correct sustained alarm for four of the five faults in at least one run, and does so in 6 of 13 fault runs. It misses most of its targets, and part of its score for two faults comes from the test day rather than the fault. Setting the targets, the checklist and the lockbox aside before modelling made those findings visible instead of hiding them behind a random split. Generating the paper from the results files also exposed a real defect, demo alarms computed with the wrong settings, which is now fixed and guarded by a test.

Data and code. The dataset is by BahooToroody et al. [3, 4], released under CC BY 4.0. The code, notebooks, results and demo are in the Keyframe repository.

References

- [1] Kjersti Aas, Martin Jullum, and Anders Løland. Explaining individual predictions when features are dependent: More accurate approximations to Shapley values. *Artificial Intelligence*, 298: 103502, 2021.
- [2] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2623–2631, 2019.
- [3] Ahmad BahooToroody, Oleksiy Bondarenko, Mohammad Mahdi Abaei, Yuta Niki, and Enrico Zio. Marine Engine Fault Dataset (version 1.0). Data set, 2026. URL <https://doi.org/10.5281/zenodo.19857425>.
- [4] Ahmad BahooToroody, Oleksiy Bondarenko, Mohammad Mahdi Abaei, Yuta Niki, and Enrico Zio. Marine Engine Fault Dataset: Open-access data under controlled reference and fault scenario conditions. Preprint, arXiv:2607.19444, 2026.
- [5] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.
- [6] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2:665–673, 2020.
- [7] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *PMLR*, pages 1321–1330, 2017.
- [8] John B. Heywood. *Internal Combustion Engine Fundamentals*. McGraw-Hill, New York, 1988.
- [9] International Organization for Standardization. ISO 14726:2008 ships and marine technology — identification colours for the content of piping systems. Standard, 2008.
- [10] Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4):15:1–15:21, 2012.

- [11] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [12] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *Proceedings of the 8th IEEE International Conference on Data Mining*, pages 413–422, 2008.
- [13] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [14] Scott M. Lundberg, Gabriel Erion, Hugh Chen, Alex DeGrave, Jordan M. Prutkin, Bala Nair, Ronit Katz, Jonathan Himmelfarb, Nisha Bansal, and Su-In Lee. From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2:56–67, 2020.
- [15] Mahdi Pakdaman Naeini, Gregory F. Cooper, and Milos Hauskrecht. Obtaining well calibrated probabilities using Bayesian binning. In *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence*, pages 2901–2907, 2015.
- [16] Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018.
- [17] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [18] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. “Why Should I Trust You?”: Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1135–1144, 2016.
- [19] David R. Roberts, Volker Bahn, Simone Ciuti, Mark S. Boyce, Jane Elith, Gurutzeta Guillera-Arroita, Severin Hauenstein, José J. Lahoz-Monfort, Boris Schröder, Wilfried Thuiller, David I. Warton, Brendan A. Wintle, Florian Hartig, and Carsten F. Dormann. Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography*, 40(8): 913–929, 2017.
- [20] Sudhir Varma and Richard Simon. Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7:91, 2006.